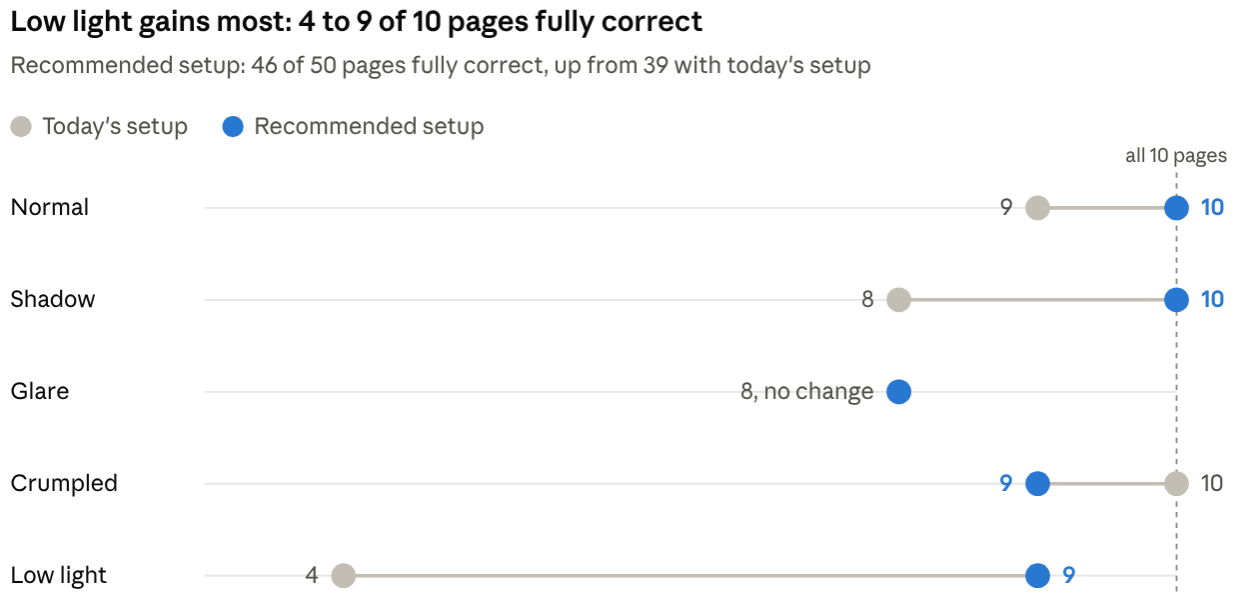
MLKit Research Spike – PPG Delivery Note Scanning

Oct 26, 2026 · @vishnu

With five small changes, MLKit reads 92% of delivery-note pages fully correct (up from 78% today) at the same speed, about 0.7 s per page. Keep MLKit, apply the changes below, and add arithmetic cross-checks for the errors it still makes in low light.

Results



Realme RMX3710 benchmark, 26 Oct 2026 · 50 images, 10 per condition · fully correct = DN number, every article and every quantity read exactly

The four pages still failing are a cut-off delivery-note number (two pages), the `68 PC` misread and one dark page; the checks under Errors tuning cannot fix should catch all four once built. Median time per page fell slightly, from 740 ms to 724 ms.

Recommendation

Keep MLKit Text Recognition v2 with the bundled Latin model, the setup the app already uses. It needs no network, which suits vessels that are offline. Make these five changes:

#	Change	Effect measured	Effort
1	Never shrink a capture below about 2400 px on the long side before OCR	Pages fully correct 78% → 92%; low-light pages 4/10 → 9/10	Small: check the capture and crop path
2	Skip per-character symbol data in `PpgOcrPlugin` for full-page OCR	−190 ms per page (−26%), no accuracy change	Done as an option: `includeSymbols: false`
3	Correct two print-format misreads before parsing: `1D` → `ID` in material numbers, and `I` or a vertical bar → `/` in page markers	Fields read correctly +5.6 points	Small: two regex rules
4	Cross-check every page with arithmetic before accepting it (see Errors tuning cannot fix)	Catches the wrong-but-valid values that remain	Medium
5	Warm up the recognizer at app start	Hides the 2.9–3.8 s first-scan delay	Small

With changes 1–3 combined, the phone read 46 of 50 pages fully correct at a median 724 ms per page, slightly faster than today's 740 ms. A 6-page note needs about 4.5 s of OCR, well inside the 1–2 minute target.

Do not add contrast boosting, black-and-white conversion or denoising. Each one made results worse overall.

What MLKit lets us control

MLKit text recognition has almost no tuning settings, so the gains come from the image we give it and the checks we run on what it returns.

Area	What we can control	What we cannot
Recognition model	Script (Latin, Chinese, Devanagari, Japanese, Korean). Bundled model, about 4 MB per script per architecture, ready offline. Or unbundled, about 260 KB, downloaded by Google Play services before first use	Confidence threshold, region of interest, language hints, character whitelist, model size, custom training or fine-tuning
Input image	Resolution, crop, rotation value, preprocessing, camera focus and exposure. Google asks for at least 16×16 px per character, with no benefit above 24×24 px	Nothing inside MLKit adapts to poor light or blur
Output	Text in blocks, lines, words and characters, each with bounding box and corner points; confidence and rotation per line, word and character	A ranked list of alternative readings for a word
Document scanner	Mode: `BASE` (camera capture, auto-rotation), `BASE_WITH_FILTER` (adds filters), `FULL` (adds image cleaning); auto or manual capture; page limit; gallery import; JPEG or PDF output	The scanner screen's layout. It needs Google Play services, downloads on first use, and needs 1.7 GB of device RAM

Two points matter for PPG. Vessel devices may be offline, so the app should keep the bundled text model it uses today. The document scanner, unlike text recognition, depends on Google Play services and a first-use download.

Licensing: MLKit is covered by the ML Kit Terms of Service, which sit under the Google APIs Terms of Service. Apache 2.0 covers only Google's sample code, not the SDK. The terms forbid reverse engineering the SDK or its models.

Sources: Text recognition v2 for Android, Document scanner for Android, GmsDocumentScannerOptions reference.

Experiments

Resolution was the setting that mattered most; skipping symbol data was the main speed gain. Every row ran the same 50 images through the same unchanged MLKit model on the same phone. "Text fixes" means the two print-format corrections applied after OCR.

Setting	Pages fully correct (of 50)	Fields read correctly	Low-light pages (of 10)	Median time per page
Recommended: 2× resolution, no symbols, text fixes	**46**	**91.7%**	**9**	**724 ms**
2× resolution + text fixes	46	91.7%	9	923 ms
1.5× resolution + text fixes	42	91.1%	4	850 ms
Today's setup + text fixes	39	88.9%	4	740 ms
Contrast boost + 1.5× resolution + text fixes	41	88.3%	5	873 ms
Today's setup, no symbols	39	83.3%	4	548 ms
Today's setup	39	83.3%	4	740 ms
Contrast boost (CLAHE)	42	82.3%	7	718 ms
Denoise + contrast boost	32	67.8%	3	731 ms
Black and white (adaptive threshold)	26	58.5%	2	730 ms

A page counts as fully correct only when its delivery-note number, every article number and every quantity were read exactly. The 2× rows stand in for a higher-resolution capture: the test photos were 1200×1600, too small for this print size, while a phone camera's own output is about 3000×4000.

Errors tuning cannot fix

Even in the best setup, MLKit sometimes returns a value that is wrong but looks valid. These need checks after OCR, not more tuning:

- **Article numbers in low light:** `475212` read as `475211`, which is a different real article on the same page; `446789` read as `448789`.
- **Quantities in low light:** `5 PC` read as `6 PC`.
- **Colour code joined to quantity:** `RED / SIGMA 6 8 PC` read as `68 PC`, so a parser would record 68 cans instead of 8.
- **Delivery-note number cut off** where its last digits touch the printed box: `5319364114` read as `53193641`.

PPG's notes carry enough redundancy to catch each of these:

Check	Example it catches
Quantity × volume per unit = total volume	`6 PC` × 16 L = 96 L, but the line says 80 L
Batch quantities add up to the line quantity	`68 PC` against batches of 5 + 3
Line quantities add up to the page-1 piece total	Any missed or wrong quantity
Delivery-note number has 10 digits and is the same on every page	`53193641`
Article and EAN pair matches the master list; EAN-13 check digit is valid	`475211` printed beside article `475212`'s EAN
Read the delivery-note barcode on page 1 with MLKit Barcode Scanning	Delivery-note number errors entirely

A line that fails a check should be flagged for the user to confirm, not rejected, so one bad read never blocks a whole check-in.

How it was tested

All numbers come from a Realme RMX3710 (MediaTek Helio G85, Android 14), a typical mid-range phone, running the app's own `ppg_ocr` channel with MLKit Text Recognition v2 16.0.1.

- **Test set:** 2 printed PPG delivery notes, 10 pages, each photographed once. Code added shadow, glare, folds and low light to each photo, giving 50 images with the printed text untouched.
- **Answer key:** every value on the 10 pages (27 rows, 2 header pages), checked on zoomed photos. Quantities match the page-1 totals and all 27 EAN-13 check digits are valid.
- **Scoring:** exact characters, no allowance for `O`/`0` or `I`/`1`. Each image ran 3 times; times are medians.

Caveats:

- 40 of the 50 images are simulated conditions, so they estimate real-world results rather than measure them.
- The source photos were compressed to 1200×1600 in transfer. Full 12 MP camera photos will take longer to decode and recognise.
- Preprocessing time was measured on a PC, not the phone.

- Accuracy is measured on MLKit's raw text. The app's parser and master-list lookup were not part of the test.

The test images, answer key, scripts and raw results are in `mlkit_research/`, a folder beside the Pixel X repository, not yet committed; the benchmark runner is `Pixorex/frontend/tool/mlkit_benchmark.dart`.

Open questions and next steps

- [ ] Re-measure speed with 2–3 full-resolution photos copied straight from the phone.
- [ ] Confirm with Suchith's document-scanner test whether `FULL` mode output reads as well as these photos, and whether a page limit above 1 lets a 6-page note be captured in one pass.
- [ ] Confirm MLKit's terms suit PPG's commercial use; the team had noted Apache 2.0, which covers only Google's sample code.
- [ ] Decide whether vessel devices will always have Google Play services, which the document scanner needs.